SOLID – Dependency Inversion Principle (DIP)

Définition

Le principe d'inversion de dépendances (Dependency Inversion Principle — DIP) est l'un des cinq principes SOLID formulés par Robert C. Martin. Il stipule que :

• Les modules de haut niveau ne doivent pas dépendre des modules de bas niveau.
• Les deux doivent dépendre d'abstractions.
• Les abstractions ne doivent pas dépendre des détails, ce sont les détails qui doivent dépendre des abstractions.

Motivation

Dans un système orienté objet classique, le code métier (haut niveau) fait souvent directement appel à des composants techniques (bas niveau), comme des fichiers, des bases de données ou des formats précis. Cela crée un couplage fort qui rend le code difficile à tester, à maintenir ou à faire évoluer.

Problème d'un design sans DIP

Lorsqu'un module de haut niveau instancie ou configure directement ses dépendances, il dépend des détails techniques. Toute modification d'une dépendance (changement de technologie, de format, etc.) nécessite alors de modifier la logique métier.

Exemple :
class ReportService { void export() { CSVExporter e = new CSVExporter(); e.write(); } }
→ Ici, ReportService dépend directement de CSVExporter.

Application du principe

Pour respecter le DIP, il faut introduire une abstraction (interface) entre le module de haut niveau et le détail technique.

interface Exporter { void write(); }
class CSVExporter implements Exporter { void write() { ... } }
class ReportService { private Exporter exporter; ReportService(Exporter e) { this.exporter = e; } void export() { exporter.write(); } }

➡ Le module ReportService dépend désormais d'une abstraction (Exporter), pas d'une implémentation concrète.

Avantages du respect du DIP

■ Réduction du couplage entre les couches du système
■ Meilleure testabilité (possibilité d'injecter des mocks ou des stubs)
■ Évolutivité accrue (changement de technologie sans toucher à la logique métier)
■ Favorise l'architecture en couches et la responsabilité claire des composants

Bonnes pratiques

• Définir des interfaces claires entre la logique métier et les composants techniques.
• Ne jamais instancier directement les dépendances concrètes dans un module métier.
• Utiliser l'injection de dépendances (constructeur, méthode, ou framework).
• Concevoir les modules autour des abstractions, et non autour des détails techniques.

Résumé

Le DIP inverse la direction naturelle des dépendances :

■ Avant : le haut niveau dépend du bas niveau
■ Après : le bas niveau dépend d'une abstraction commune

En respectant le DIP, le code devient plus modulaire, plus stable et plus résistant aux changements techniques.